



ENERGY · SOLARE · MOBILITÀ ELETTRICA

Tesla Energy Controller

Ricarica intelligente dell'auto elettrica con l'energia fotovoltaica in eccesso, governata da una Raspberry Pi che integra l'impianto **SolarEdge**, il sistema domotico **VIMAR** e la **Tesla**.

AMBITO

Autoconsumo solare
Ricarica EV controllata

PIATTAFORMA

Raspberry Pi · edge
Python · tutto in locale

DOCUMENTO

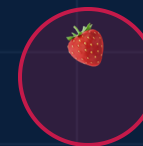
Presentazione tecnica
Architettura e funzionamento



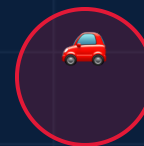
SolarEdge



VIMAR



Raspberry



Tesla

Un servizio software che **campiona di continuo** quanta energia produce il fotovoltaico, quanta ne consuma la casa e lo stato dell'auto in una finestra in RAM; **ogni 5 minuti** ne calcola la media (EWMA) e adatta **la corrente di ricarica** per assorbire il più possibile dal sole, riducendo i prelievi dalla rete.

- **Obiettivo:** massimizzare l'autoconsumo dell'energia solare caricando l'auto con i kW che altrimenti finirebbero in rete.
- **Non invasivo:** di norma regola solo l'amperaggio di una ricarica *già avviata*; solo la logica **Quota Potenza** può sospendere/riprendere per rispettare il limite quartorario.
- **Sicuro per default:** parte in modalità **dry-run** — calcola la decisione ma non invia comandi finché non è validato sul campo.

Le fonti di dato

Ogni ciclo combina letture indipendenti e locali:



SolarEdge — produzione FV istantanea
Modbus TCP · API cloud · portale web



Contatore ALFA Sinapsi — import/export di rete
Modbus TCP · base della Quota Potenza



Gateway VIMAR By-me — consumi della casa
misura per singolo circuito, via rete locale



Tesla — stato e potenza di ricarica
Bluetooth LE ufficiale o Wall Connector

5min

media EWMA e
decisione

690w

per ogni ampere
(3×230 V)

≈5w

consumo della
Raspberry

IL PROBLEMA

L'energia solare in eccesso viene "regalata" alla rete

- Nelle ore centrali il FV produce più di quanto la casa consuma: il surplus viene immesso in rete a un valore basso.
- La Tesla è un grande accumulatore: caricarla con quel surplus vale molto più che venderlo.
- Caricare a corrente fissa spreca sole quando c'è poco, o preleva dalla rete quando la casa accende i carichi pesanti (pompa di calore, clima, forno...).
- Per dosare bene serve sapere **in tempo reale quanto consuma davvero la casa** — ed è qui che entra VIMAR.

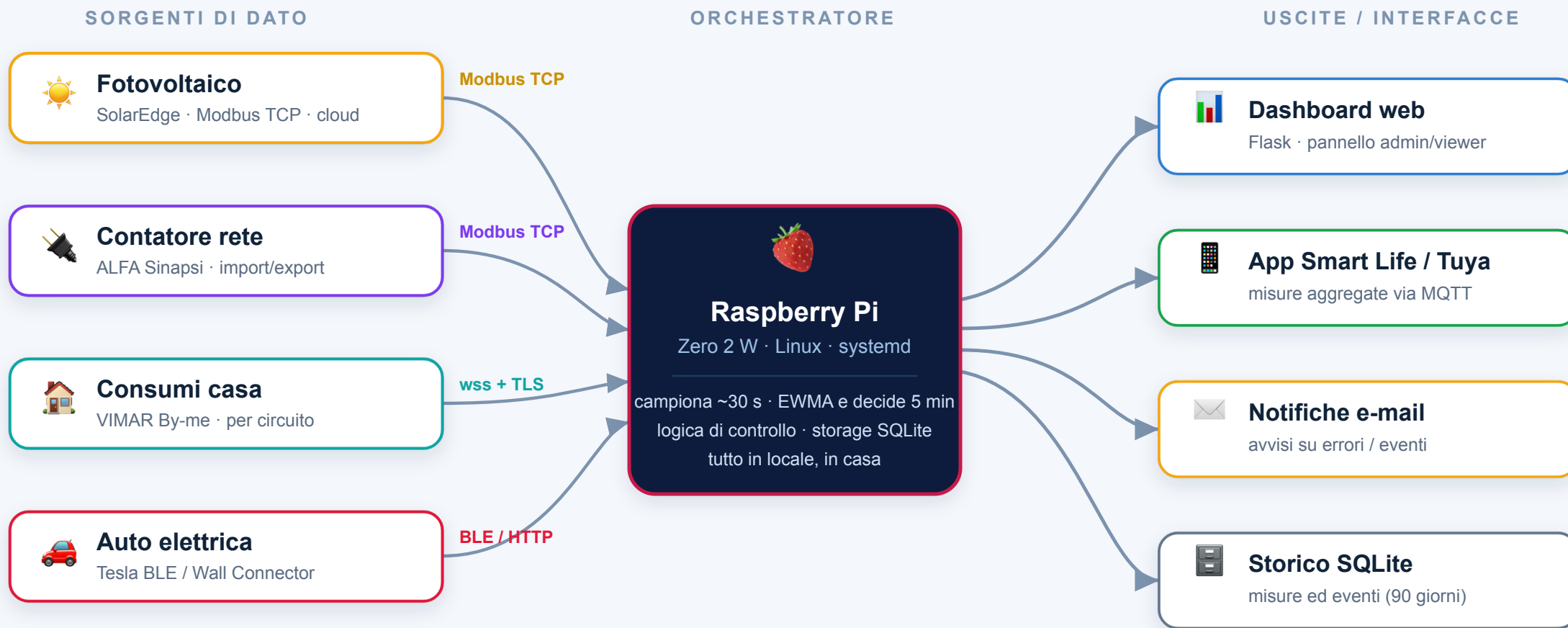
L'OBIETTIVO

Inseguire il sole, rispettando la casa

- **Autoconsumo prima di tutto:** assegnare alla Tesla la potenza fotovoltaica disponibile, al netto dei consumi domestici letti da VIMAR.
- **Quota potenza:** con il contatore ALFA, restare sotto la potenza media quartoraria contrattuale (limite hard); il solare più una quota extra di rete ammessa alza la corrente.
- **Stabilità:** variazioni morbide (max +2 A per ciclo, isteresi) per non stressare auto e impianto.
- **Trasparenza:** ogni decisione registrata, visibile su una dashboard e sull'app Smart Life.

La regola di base

$$\text{ampere} = \lfloor (\text{FV} - \text{consumi casa} + \text{extra ammesso}) / (\text{tensione} \times \text{fasi}) \rfloor$$



Tutte le comunicazioni con auto e impianti sono **locali e cifrate**; nessun dato sensibile lascia la rete di casa per funzionare.

Il ruolo del gateway VIMAR

Il sistema si collega al **gateway VIMAR By-me** come applicazione di terze parti, sfruttando l'**API IP locale** documentata. Da lì ottiene la misura energetica della casa, suddivisa **per singolo circuito**.

Il dialogo con il gateway, passo per passo

1. **Scoperta** del gateway in rete via mDNS `_vimar-devctrl._tcp` (IP, modello, versioni).
2. **Sessione** sicura WebSocket su TLS — `wss:// porta 20615`, protocollo 2.7.
3. **Abbinamento** una tantum con *Third-Party Tag* e *Setup Code*; il gateway restituisce le credenziali permanenti.
4. **Autenticazione** a ogni accesso firmando la sfida con la **chiave privata RSA** (PKCS#1 v1.5).
5. **Lettura** tramite `sfdiscovery` dei punti "Energy" con i relativi valori istantanei.

GRANDEZZE LETTE DALL'IMPIANTO

`SFE_State_GlobalActivePowerConsumption`

Potenza assorbita → **consumo casa**

`SFE_State_GlobalActivePowerProduct`

Potenza prodotta (se monitorata)

`SFE_State_GlobalActivePowerExchange`

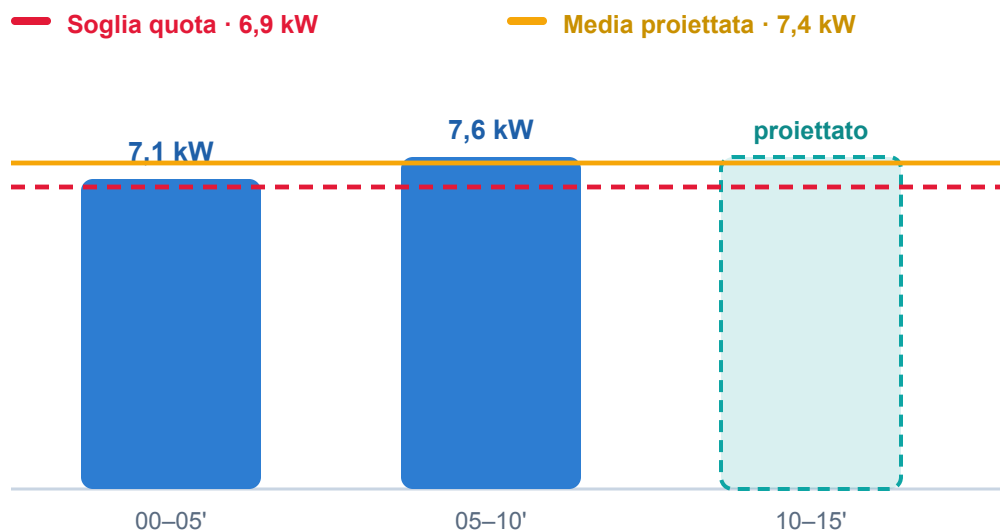
Scambio con la rete (import/export)

☒ **Impianto trifase** 230 V ☐ misura per singolo circuito

La chiave privata e il certificato CA VIMAR restano **solo sulla Raspberry**, con permessi di file ristretti. L'app legge soltanto le misure: non comanda i carichi domestici.

Quota Potenza: la media quartoraria

Un quarto d'ora = tre slot da 5 minuti



Le letture ALFA vengono aggregate in tre slot da 5 minuti allineati al quarto d'ora. Se la **proiezione** supera la soglia contrattuale, il controller riduce la corrente — o sospende — senza attendere l'isteresi.

Come rispetta il limite

1. **Aggrega** le letture in tre slot da 5 minuti.
2. **Proietta** gli slot mancanti tenendo il prelievo più recente.
3. **Confronta** con la soglia: se la supera, taglia la corrente.
4. **Sospende e riprende** se serve — scelta economica, non protezione.

📈 In dashboard è evidenziata la **Quota Potenza**: la massima media quartoraria **completa** raggiunta nel mese.

Una piccola **Raspberry Pi Zero 2 W**, collocata **vicino all'auto**, è il cervello sempre acceso del sistema: parla con tutti i dispositivi e decide in autonomia, senza dipendere dal cloud.

- **Sempre vicina alla Tesla:** la portata del Bluetooth richiede prossimità; per questo la Pi sta in garage/box, non in centralino.
- **Due servizi Linux autonomi** (systemd): il controller+dashboard e il bridge Tuya, riavviati automaticamente in caso di problemi.
- **Leggera e silenziosa:** nessun ventilatore, ~5 W, gira 24/7 su una scheda da 512 MB di RAM.
- **Tutto in locale:** letture, storico e decisioni restano in casa; il cloud serve solo per notifiche e app, se attivati.

Cosa fa ad ogni ciclo (ogni 5 minuti)

- **Legge SolarEdge**
produzione FV
- **Legge VIMAR + ALFA**
consumi casa e scambio di rete
- **Legge la Tesla**
BLE o Wall Connector: stato, tensione, fasi
- **Media EWMA e salva**
media 5 min + snapshot SQLite
- **Regola l'amperaggio**
solo se necessario e consentito

2servizi systemd
autonomi**90**ggstorico misure
conservato**0**dipendenze dal cloud
per decidere

Come comunica con l'auto

- Usa `tesla-control`, lo strumento **ufficiale Tesla** per i veicoli, tramite **Bluetooth LE**.
- **Nessun account, password Tesla o cloud:** il BLE non richiede Fleet API, OAuth né dominio pubblico.
- Abbinamento sicuro: una chiave privata generata sulla Pi e autorizzata in auto appoggiando la **card NFC** Tesla, con il ruolo minimo `charging_manager`.
- La chiave privata **resta solo sulla Raspberry**: senza di essa nessuno può comandare l'auto.
- In alternativa la potenza Tesla si legge dal **Wall Connector Gen 3** locale (HTTP), senza svegliare l'auto.

Cosa fa — e cosa non fa

- ✓ **Legge** stato di carica, tensione, corrente e numero di fasi (comando `state charge`).
- ✓ **Imposta** solo l'amperaggio di ricarica (`charging-set-amps`).
- ✗ **Di norma non** avvia né ferma la carica e **non** sveglia l'auto; solo la **Quota Potenza** può sospendere/riprendere.

Prima di agire verifica che l'auto sia sveglia e in carica, che l'orologio sia sincronizzato (NTP) e che fasi e tensione siano nei limiti. Gli errori BLE sono classificati (distanza, Bluetooth, chiave, orologio, stato auto) per la diagnosi a distanza.

230 V × 3

trifase → 690 W per ampere

1–16 A

range gestito · minimo configurabile fino a 1 A

2 cmd

massimo per ciclo · chiamate BLE serializzate

NFC

abbinamento una tantum con card Tesla

Catena di verifiche prima di ogni comando



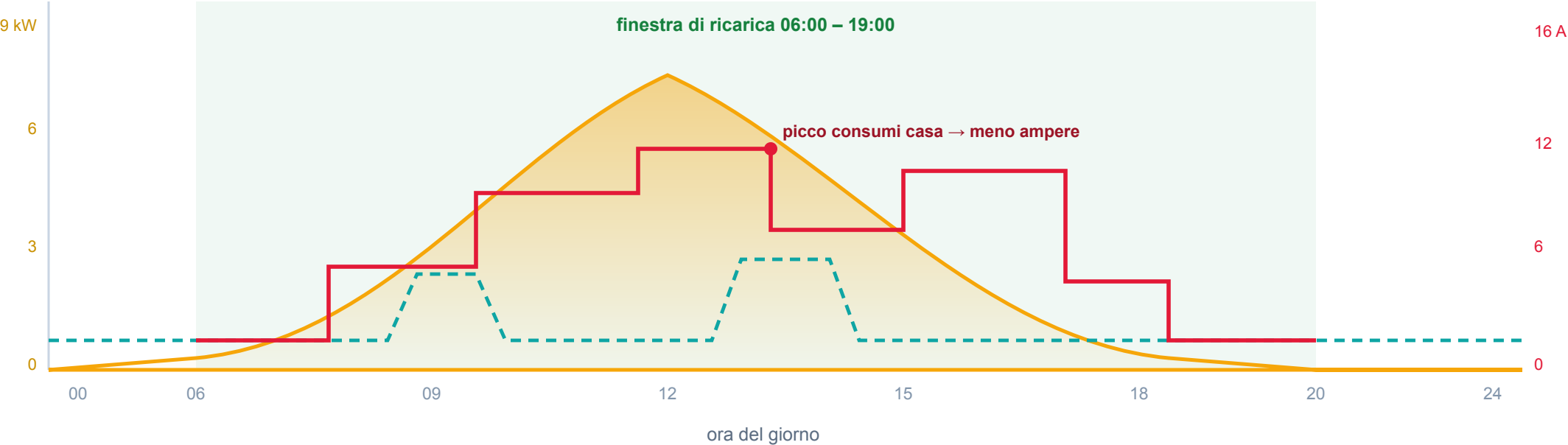
aggiunge max +2 A/ciclo,
rispetta la quota potenza

Le reti di sicurezza

- **Dry-run di default:** calcola ma non invia, finché non è validato sul campo.
- **Salita lenta, discesa immediata:** +2 A massimo per ciclo, riduzione subito.
- **Isteresi 1 A:** niente comandi per micro-variazioni.
- **Quota potenza & override manuale:** resta sotto la soglia quartoraria ALFA; se l'utente alza la corrente, il sistema si fa da parte.
- **Fail-safe:** a ogni errore di rete o dato, nessun comando viene inviato.

⚠ **Importante:** è un controllo applicativo, **non** sostituisce le protezioni elettriche (magnetotermico, differenziale) né un load-balancing certificato.

Produzione FV e amperaggio Tesla nell'arco della giornata



■ Produzione fotovoltaica (SolarEdge) ■ Consumo casa (VIMAR) ■ Corrente di ricarica Tesla (A) ■ Finestra di ricarica

Esempio illustrativo del comportamento atteso: la corrente Tesla insegue il sole ma si abbassa quando la casa accende i carichi pesanti, mantenendo il prelievo dalla rete entro il limite.

Snapshot a mezzogiorno

Esempio illustrativo · ore 12:00

Dove va l'energia prodotta dal sole

Sorgenti



Utilizzi



Bilancio: **quasi tutta** la ricarica arriva dal sole; poca energia dalla rete (**export quasi nullo**).

Ripartizione dei consumi



+0,9kW

export mattutino tipico
prima dell'avvio carichi

Il software è stato progettato e scritto in **programmazione assistita dall'intelligenza artificiale**: due assistenti di coding hanno lavorato come "colleghi", sotto guida e revisione umana.



Claude · Anthropic (Claude Code)

- Architettura a moduli, modello dati e **logica di controllo** con tutte le sicurezze.
- **Reverse engineering** dei protocolli VIMAR (API IP, firma RSA), SolarEdge e contatore ALFA.
- Suite di **test automatici** e gestione diagnostica degli errori.



Codex · OpenAI

- Sviluppo iterativo di funzioni, **script di deploy** e provisioning Raspberry.
- Integrazione bridge **Tuya/Smart Life** (MQTT) e pannello web.
- Rifiniture, controllo incrociato del codice e correzioni rapide.

METODO: SPECIFICA → TEST → IMPLEMENTAZIONE → COLLAUDO DRY-RUN

21

moduli Python (~8.700 righe)

157

test automatici · 9 suite

11

script di deploy / provisioning

6

sistemi integrati (FV · rete · casa · auto · cloud)



Dati e credenziali

- Chiavi VIMAR, Tesla e credenziali (ALFA, SolarEdge) solo sulla Pi, permessi file ristretti (0600).
- VIN, password e token **mai mostrati** in dashboard né nei report.
- Tutte le connessioni locali cifrate (TLS / BLE autenticato).



Accesso e rete

- Dashboard in ascolto solo in locale; accesso da remoto via **tunnel SSH**.
- Due ruoli: **admin** (configura) e **viewer** (osserva e accende/spegne).
- Storico cancellato automaticamente dopo 90 giorni.



Robustezza operativa

- Ritenta gli errori transitori (BLE/rete) con attese crescenti.
- In caso di guasto persistente invia un **report e-mail** diagnostico.
- Servizi riavviati da systemd; il monitoraggio continua anche a controller spento.



Confine chiaro di responsabilità. Il sistema è un **controllo applicativo** dell'energia: ottimizza la ricarica, ma non sostituisce in alcun modo le protezioni elettriche dell'impianto, il dimensionamento dei cavi o un load-balancing certificato. L'avvio e l'arresto della ricarica restano sempre **manuali**.



STATO DEL PROGETTO · ROADMAP

Pronto per l'**esercizio**

GIÀ ATTIVO OGGI

- **Raspberry Pi** collegata a SolarEdge, contatore ALFA, VIMAR e Tesla.
- **Dashboard web**, **Quota Potenza** e bridge Smart Life operativi.
- Lettura **per circuito** e snapshot energetici storicizzati in SQLite.
- Funzionamento in **dry-run** per validazione sul campo.

PROSSIMI PASSI

- Passaggio a **modalità live** dopo conferma di segno potenza, fasi e decisioni.
- Validazione dei registri diagnostici **ALFA** (fasce, energie F1/F2/F3).
- Affinamento della finestra solare **stagionale** (alba/tramonto).

PROGETTO

Tesla Energy Controller

Autoconsumo solare · edge

PIATTAFORMA

Raspberry Pi · Python

Tutto in locale, cifrato

GRAZIE

Domande?

Demo e documentazione tecnica